

GROUP PROJECT

- Module: **CST2550 Software Engineering Management and Development**

Pseudocode: -

Postal Service Management System

Algorithm 1 — Add Parcel to Hash Table

Purpose:

- To add a new parcel into the Hash Table using its TrackingID as the key.

Pseudocode:

```
-> FUNCTION AddParcel(parcel):  
    index = HashFunction(parcel.TrackingID)  
    IF bucket[index] is empty THEN  
        bucket[index] = parcel  
    ELSE  
        ADD parcel to linked list at bucket[index]  
    END IF
```

-> END FUNCTION

- Explanation:

- The HashFunction converts the TrackingID into an index number. The parcel is then stored at that index in the bucket array. If two parcels hash to the same index (collision), they are stored in a linked list at that bucket.
-

Algorithm 2 — Search Parcel in Hash Table

Purpose:

- To find a parcel in the Hash Table using its TrackingID.

Pseudocode:

```
-> FUNCTION SearchParcel(trackingID):  
    index = HashFunction(trackingID)  
    current = bucket[index]  
    WHILE current is not null DO  
IF current.TrackingID == trackingID THEN  
        RETURN current  
    END IF  
        current = current.next  
    END WHILE  
RETURN null
```

-> END FUNCTION

- Explanation:

- The TrackingID is hashed to find the correct bucket. The linked list at that bucket is then traversed until the matching parcel is found. If not found, null is returned.
-

Algorithm 3 — Delete Parcel from Hash Table

Purpose:

- To remove a parcel from the Hash Table using its TrackingID.

Pseudocode:

-> prev = null

current = bucket[index]

WHILE current is not null DO

 IF current.TrackingID == trackingID THEN

 IF prev is null THEN

 bucket[index] = current.next

 ELSE

 prev.next = current.next

```
        END IF
        RETURN
    END IF
    prev = current
    current = current.next
END WHILE

-> END FUNCTION
```

- Explanation:

- The TrackingID is hashed to find the correct bucket. If the parcel is at the head of the linked list, it is removed directly. Otherwise, the list is traversed to find and remove the matching node.
-

Algorithm 4 — Insert Parcel into BST

Purpose:

- To insert a parcel into the Binary Search Tree in the correct sorted position by TrackingID.

Pseudocode:

```
-> FUNCTION BSTInsert(node, parcel):  
IF node is null THEN  
    RETURN new Node(parcel)  
END IF  
IF parcel.TrackingID < node.TrackingID THEN  
    node.left = BSTInsert(node.left, parcel)  
ELSE  
    node.right = BSTInsert(node.right, parcel)  
END IF  
RETURN node  
-> END FUNCTION
```

- Explanation:

- Starting at the root, the TrackingID is compared at each node. If smaller it goes left, if larger it goes right. This continues until an empty spot is found where the new parcel is inserted.

Algorithm 5 — Enqueue Delivery into Queue

Purpose:

- To add a new delivery to the back of the Queue.

Pseudocode:

```
-> FUNCTION Enqueue(delivery):  
newNode = new Node(delivery)  
IF front is null THEN  
front = newNode  
rear = newNode  
ELSE  
rear.next = newNode  
rear = newNode  
END IF  
count = count + 1  
-> END FUNCTION
```

- **Explanation:**
 - A new node is created for the delivery. If the queue is empty both front and rear point to the new node. Otherwise, the new node is added to the rear of the queue. This follows First In First Out (FIFO) order.
-

Algorithm 6 — Dequeue Delivery from Queue

Purpose:

- To remove and return the delivery at the front of the Queue.

Pseudocode:

```
-> FUNCTION Dequeue():  
    IF queue is empty THEN  
        RETURN null  
    END IF  
    temp = front  
    front = front.next  
    IF front is null THEN  
        rear = null  
    END IF  
    count = count - 1  
    RETURN temp.data  
-> END FUNCTION
```

- Explanation:

- If the queue is empty null is returned. Otherwise, the front node is removed, and the next node becomes the new front. If the queue becomes empty after removal both front and rear are set to null.
-

Algorithm 7 — Hash Function

Purpose:

- To convert a TrackingID string into a valid index number for the bucket array.

Pseudocode:

```
-> FUNCTION HashFunction(trackingID):  
    hash = 0  
    FOR each character c in trackingID DO  
        hash = hash + ASCII value of c  
    END FOR  
    index = hash MOD size  
    RETURN index  
-> END FUNCTION
```

- Explanation:

- Each character in the TrackingID is converted to its ASCII value and added together to create a hash number. The MOD operator ensures the index stays within the bounds of the bucket array. This guarantees a valid array position is always returned.
-

Algorithm 8 — Search Parcel in BST

Purpose:

- To find a parcel in the Binary Search Tree using its TrackingID.

Pseudocode:

```
-> FUNCTION BSTSearch(node, trackingID):  
    IF node is null THEN  
        RETURN null  
    END IF  
    IF node.TrackingID == trackingID THEN  
        RETURN node  
    END IF  
    IF trackingID < node.TrackingID THEN  
        RETURN BSTSearch(node.left, trackingID)  
    ELSE  
        RETURN BSTSearch(node.right, trackingID)  
    END IF  
-> END FUNCTION
```

- Explanation:

- Starting at the root the TrackingID is compared at each node. If it matches the parcel is returned. If smaller the search goes left, if larger it goes right. This halves the search space at each step giving $O(\log n)$ average time complexity. If not found null is returned.

Algorithm 9 — In Order Traversal of BST

Purpose:

- To visit every node in the BST and return all parcels in sorted order by TrackingID.

Pseudocode:

```
-> FUNCTION InOrderTraversal(node):  
    IF node is null THEN  
        RETURN  
    END IF  
    InOrderTraversal(node.left)  
    PRINT node.parcel  
    InOrderTraversal(node.right)  
-> END FUNCTION
```

- Explanation:

- The traversal visits the left subtree first, then the current node, then the right subtree. Because of how the BST is built this produces parcels in ascending order by TrackingID. This is used to display sorted parcel lists to the user. This operation runs at $O(n)$ as every node is visited once.
-

Algorithm 10 — Register New User

Purpose:

- To register a new user into the system by validating their details and saving them to the database.

Pseudocode:

```
-> FUNCTION RegisterUser(user):  
    IF user.Password != user.ConfirmPassword THEN  
        RETURN "Passwords do not match"  
    END IF  
    IF UserExists(user.Email) THEN  
        RETURN "Email already registered"  
    END IF  
    user.UserID = GenerateUserID()  
    user.CreatedDate = GetCurrentDate()  
    INSERT user INTO database  
    RETURN "Account created successfully"  
-> END FUNCTION
```

- Explanation:

- The system first checks if the passwords match. Then it checks if the email is already registered. If both checks pass a unique UserID is auto generated and the current date is recorded. The user details are then saved to the database and a success message is returned.
-

Algorithm 11 — Login User

Purpose:

- To authenticate a user by checking their email and password against the database.

Pseudocode:

```
-> FUNCTION LoginUser(email, password):  
    user = SearchUserByEmail(email)  
    IF user is null THEN  
        RETURN "Email not found"  
    END IF  
    IF user.Password != password THEN  
        RETURN "Incorrect password"  
    END IF  
    RETURN user  
-> END FUNCTION
```

- Explanation:

- The system searches the database for a user with the given email. If no user is found an error message is returned. If the user is found the password is checked. If the password matches the user object is returned and the user is logged in successfully.
-

Algorithm 12 — Add Customer

Purpose:

- To add a new customer to the system and initialise their parcel tracking.

Pseudocode:

```
-> FUNCTION AddCustomer(customer):  
    IF CustomerExists(customer.Email) THEN  
        RETURN "Customer already exists"  
    END IF  
    customer.CustomerID = GenerateCustomerID()  
    INSERT customer INTO database  
    parcelQueue = new Queue ()  
    RETURN "Customer added successfully"  
-> END FUNCTION
```

- Explanation:

- The system first checks if a customer with the same email already exists to prevent duplicates. If not a unique CustomerID is auto generated and the customer is saved to the database. A new Queue is initialised to track the customer's parcels in FIFO order.
-

Algorithm 13 — Assign Driver to Delivery

Purpose:

- To assign a driver and route to a delivery record in the system.

Pseudocode:

```
-> FUNCTION AssignDriver(deliveryID,  
                           driverName, route):  
    delivery = SearchDelivery(deliveryID)  
    IF delivery is null THEN  
        RETURN "Delivery not found"  
    END IF  
    delivery.DriverName = driverName  
    delivery.Route = route  
    delivery.AssignedDate = GetCurrentDate()  
    delivery.DeliveryStatus = "Assigned"  
    UPDATE delivery IN database  
    RETURN "Driver assigned successfully"  
-> END FUNCTION
```

- Explanation:

- The system finds the delivery record by its ID. The driver's name, route and assigned date are updated. The status is changed to Assigned and the record is saved back to the database.
-

Algorithm 14 — Update Delivery Status

Purpose:

- To update the status of a delivery as it progresses through the system.

Pseudocode:

```
-> FUNCTION UpdateDeliveryStatus(deliveryID,
newStatus):
    delivery = SearchDelivery(deliveryID)
    IF delivery is null THEN
        RETURN "Delivery not found"
    END IF
    IF newStatus == "Failed" THEN
        delivery.AttemptCount = delivery.AttemptCount + 1
        delivery.LastAttemptDate = GetCurrentDate()
        IF delivery.AttemptCount >= 3 THEN
            delivery.DeliveryStatus = "Return to Depot"
            UPDATE delivery IN database
            RETURN "Max attempts reached"
        END IF
    END IF
    delivery.DeliveryStatus = newStatus
    UPDATE delivery IN database
    RETURN "Status updated successfully"
-> END FUNCTION
```

- Explanation:

- The system finds the delivery by its ID. If the new status is Failed the attempt count is incremented and the last attempt date is recorded. If the attempt count reaches 3 the parcel is returned to depot. Otherwise, the status is updated and saved.
-

Algorithm 15 — Get Dashboard Statistics

Purpose:

- To count parcels by status for display on the Dashboard screen.

Pseudocode:

```
-> FUNCTION GetDashboardStats():  
    totalParcels = COUNT all parcels  
    inTransit = COUNT parcels WHERE  
        Status == "In Transit"  
    deliveredToday = COUNT parcels WHERE  
        Status == "Delivered" AND  
        DateSent == today  
    failed = COUNT parcels WHERE  
        Status == "Failed"  
    RETURN totalParcels, inTransit,  
        deliveredToday, failed  
-> END FUNCTION
```


- **Explanation:**

- The system counts all parcels in the Hash Table grouped by their status. These counts are returned to the Dashboard screen to display the 4 stat cards Total Parcels, In Transit, Delivered Today and Failed.
-

Algorithm 16 — Search Customer

Purpose:

- To find a customer in the database by their name, email or postcode.

Pseudocode:

```
-> FUNCTION SearchCustomer(searchTerm):  
    results = empty list  
    FOR each customer IN database DO  
        IF customer.FullName contains searchTerm  
        OR customer.Email contains searchTerm  
        OR customer.PostCode contains searchTerm  
        THEN  
            ADD customer to results  
        END IF  
    END FOR  
    IF results is empty THEN
```

```
        RETURN "No customers found"
    END IF
    RETURN results
-> END FUNCTION
```

- Explanation:

- The system loops through all customers in the database and checks if the search term matches the name, email or postcode. All matching customers are added to a results list which is returned to the Customer Details screen.
-

Algorithm 17 — Delete Customer

Purpose:

- To remove a customer and their associated parcels from the system.

Pseudocode:

```
-> FUNCTION DeleteCustomer(customerID):
    customer = SearchCustomerByID(customerID)
    IF customer is null THEN
        RETURN "Customer not found"
    END IF
    parcels = GetParcelsByCustomer(customerID)
    FOR each parcel IN parcels DO
```

```
                DeleteParcel(parcel.TrackingID)
            END FOR
        DELETE customer FROM database
        RETURN "Customer deleted successfully"
-> END FUNCTION
```

- Explanation:

- The system first finds the customer by their ID. It then retrieves all parcels linked to that customer and deletes them one by one from the Hash Table and database. Finally, the customer record itself is deleted. This ensures no orphaned parcel records remain.
-

Algorithm 18 — Update Parcel Status

Purpose:

- To update the status of a parcel as it moves through the delivery journey on the Track Parcel screen.

Pseudocode:

```
-> FUNCTION UpdateParcelStatus(trackingID,
                                newStatus):
    parcel = SearchParcel(trackingID)
    IF parcel is null THEN
        RETURN "Parcel not found"
```

```
END IF
IF newStatus == "Delivered" THEN
    parcel.Status = "Delivered"
    UPDATE parcel IN database
delivery = SearchDeliveryByTrackingID(trackingID)
    UPDATE delivery IN database
    RETURN "Parcel delivered successfully"
END IF
    parcel.Status = newStatus
    UPDATE parcel IN database
    RETURN "Status updated successfully"
-> END FUNCTION
```

- Explanation:

- The system searches for the parcel using its TrackingID in the Hash Table. If found the status is updated to the new value. If the status is Delivered both the parcel and delivery records are updated together. This covers the journey from Pending all the way through to Delivered on the Track Parcel screen.

Algorithm 19 — Auto Assign Delivery

Purpose:

- To automatically create a delivery record when a new parcel is added to the system.

Pseudocode:

```
-> FUNCTION AutoAssignDelivery(parcel):  
    delivery = new Delivery()  
    delivery.DeliveryID = GenerateDeliveryID()  
    delivery.TrackingID = parcel.TrackingID  
    delivery.DeliveryStatus = "Assigned"  
    delivery.AttemptCount = 0  
    delivery.AssignedDate = GetCurrentDate()  
    INSERT delivery INTO database  
    Enqueue(delivery)  
    RETURN "Delivery created successfully"  
  
-> END FUNCTION
```

- Explanation:

- When a new parcel is added to the system this function automatically creates a matching delivery record. A unique DeliveryID is generated and linked to the parcel via TrackingID. The delivery is added to the database and added to the delivery Queue for processing in FIFO order. This covers the auto assigns delivery flow in the flowchart.

Algorithm 20 — Logout User

Purpose:

- To log out the current user and return them to the Login screen.

Pseudocode:

```
-> FUNCTION LogoutUser():  
    currentUser = null  
    sessionToken = null  
    ClearSessionData()  
    RETURN "Logged out successfully"  
-> END FUNCTION
```

- Explanation:

- The system clears the current user object and session token from memory. All session data is cleared to prevent unauthorised access. The user is then redirected back to the Login screen. This covers the Logout => Login flow in the flowchart.